

⚠ Мы не можем доставить сообщение по указанному вами адресу e-mail. Чтобы разобраться, в чём дело, перейдите, пожалуйста, по ссылке.



StonedCatt 7 минут назад

Минимальный продакшн-шаблон для Next.js приложения

🟢 Простой ⌚ 7 мин 👁 134

ReactJS*, Open source*, GitHub*, Программирование*

Кейс



Введение

Я Full-stack разработчик, занимаюсь веб-разработкой примерно с 2014 года. В последние годы чаще занимаюсь внутренними проектами, внешними заказами и своими микропродуктами. Постоянная боль - нехватка удобного, желательно бесплатного инструмента, чтобы развернуть очередной новый проект под задачу без лишней возни с инфраструктурой.

За несколько лет удалось довести до рабочего состояния наработки по:

Все потоки



Курсы для карьерного роста

+1



- выдаче и обновлению SSL-сертификатов для домена;
- минимальному набору метрик (Grafana, Prometheus, Loki и др.) — по желанию, через флаги.

В итоге получились два проверенных временем бойлерплейта — Nest.js и Next.js. В этой статье хочу поделиться вторым. Чтобы не усложнять инфраструктуру, в шаблоне заложен один стек: Next.js, авторизация и работа с БД в рамках одного приложения.

Возможно, такой шаблон уже не диковина, но если он сэкономит кому-то часы на настройке деплоя и окружения — буду рад.

От идеи до релиза

Почти каждому разработчику хочется довести свою идею до работающего продукта — пет-проект, микропродукт, внутренний сервис. Я буду говорить от своего опыта: у всех задачи и

контекст разные.

Сейчас многое можно собрать с помощью ИИ и готовых туториалов. Но довести продукт до стабильного релиза и не застрять на инфраструктуре — отдельная история. Поэтому я и выложил свой вариант standalone-шаблона на Next.js: в нём уже решены деплой, сертификаты и опционально — метрики. Вы можете взять его как основу и спокойно заниматься логикой приложения.

С шаблоном можно делать что угодно: он открыт, в репозитории — минимальный набор: авторизация, работа с куками и БД, скрипты и воркфлоу под ваш сервер.

Что внутри

Шаблон заточен под один сценарий: один репозиторий, один стек (Next.js + API routes + при необходимости БД и сессии). Никакой отдельной «админки» по инфраструктуре — всё управляется конфигами и флагами в GitHub Actions.

Деплой и окружения

- Workflow под stage и prod (отдельные файлы в `.github/workflows/`), общая логика вынесена в reusable workflow.
- Поддержка режима «сборка на сервере» и режима «образ из GHCR»; при желании — blue/green (с нюансами).
- Деплой идёт по push в нужную ветку; на сервере поднимаются только нужные контейнеры.

Сертификаты и nginx

- Let's Encrypt через certbot: первичная выдача и продление.
- Nginx как единая точка входа: проксирование на Next.js, при включённых метриках — подпуть под Grafana. Конфиг nginx собирается из шаблонов с учётом флага `METRICS_ENABLED` : если метрики выключены, блоки Grafana в конфиг не попадают, и nginx не зависит от контейнера Grafana.

Опциональные части

- **Redis** — поднимается только при `redis_enabled: true` в конфиге воркфлоу.
- **Метрики (Grafana, Prometheus, Loki, Telegraf, Promtail и т.д.)** — только при `metrics_enabled: true` . По умолчанию оба флага выключены: всегда поднимаются только nginx и приложение.
- **Mongo** - поднимается только при `mongo_enabled: true` в конфиге воркфлоу. Именно эта БД на данный момент идет в шаблоне из коробки. Вы всегда можете отказаться от части логики связанной с внутренней БД для Nextjs приложения. Особенно если речь идет о разделении финального бэкенда от логики клиентского приложения.

Так вы можете стартовать с минимального набора (приложение + nginx + сертификаты) и при необходимости включить кэш и дашборды одной правкой в `stage-deploy.yml` / `prod-deploy.yml` .

Авторизация и БД

- В репозитории заложен минимальный слой для авторизации и работы с БД в рамках Next.js (API routes, куки, сессии). Если вам нужен отдельный бэкенд — те же воркфлоу и скрипты можно оставить, а логику в `app/api/[...]/route` и флаги деплоя подстроить под свою связку.

Для кого это может быть полезно

Шаблон хорошо подходит для пет-проектов, внутренних сервисов и MVP. Тем, кто хочет быстро вывести идею в прод без погружения в DevOps: один репо, понятные флаги, один сервер. И тем, кто уже привык к Next.js и хочет иметь под рукой заготовку с деплоем и, при желании, метриками — без лишних сервисов по умолчанию.

Репозиторий: [nextjs-super-boilerplate](#). Если такой подход вам близок — можете использовать шаблон как основу для своего следующего проекта или быстрого релиза; буду рад звёздочке и обратной связи. Так же обязательно создавайте issues, если у Вас возникают трудности или появляются баги работы базовой архитектуры.

Помимо шаблона деплоя

В репозитории есть не только конфиги и скрипты под сервер: внутри — полноценный пример клиентского приложения. Реализован сценарий с кабинетом и аутентификацией: вход по email/паролю, регистрация, защищённые маршруты, получение профиля с сервера и работа с сессией через куки. Этим можно пользоваться как референсом или сразу как основой под свой продукт.

Плюс в кодовой базе заложен свой UI-кит: кнопки, поля ввода, типографика, сайдбар, баджи и другие переиспользуемые компоненты. Они используются на страницах примера (в том числе на демо-странице UI Kit) и задают единый стиль. Если ваш стек — React и Next.js, вы можете опереться на этот набор компонентов и донастроить его под свой дизайн, не начиная интерфейс с нуля.

Для тех, кто дочитал до конца

Как это все использовать

Есть несколько базовых способов:

Клонирование

```
git clone https://github.com/Fedorrychkov/nextjs-super-boilerplate.git your-project-name
```

Объяснить с  SourceCraft

При таком варианте сохраняется изначальная история коммитов; дальше нужно указать свой `origin` (ваш новый репозиторий) и пушить уже туда.

Форк

Перейдите в [репозиторий](#) и нажмите кнопку **Fork**. У вас появится копия у себя в аккаунте, с которой можно работать и в которую можно пушить.

Использовать репозиторий как шаблон

На странице репозитория нажмите **Use this template** (кнопка рядом с Fork). Создастся новый репозиторий в вашем GitHub без истории коммитов — удобно сразу начать свой проект с чистой истории.

Локальный запуск проекта

Я использую `pnpm` для локальной работы, но должно работать всё то же самое через `npm` и `yarn`. В Docker-конфигах используется `npm`. Файла `package-lock` в репозитории нет — только `pnpm-lock.yaml`.

После клонирования или создания репо из шаблона:

1. Установите зависимости: `pnpm install` (или `npm install / yarn`).
2. Скопируйте `.env.example` в `.env.local` и при необходимости подставьте свои значения (JWT, MongoDB и т.д.).
3. Если нужна локальная MongoDB в контейнере: `make up-local` (поднимает mongo из `docker-compose.dev.yml`). Подключение — по полям из `.env.local` (для контейнера укажите `MONGO_HOST=localhost`).
4. Запуск dev-сервера: `pnpm run dev:local` (или `npm run dev:local`). Приложение откроется по адресу из вывода (обычно <http://localhost:3000>).

Остановить локальный mongo: `make down-local`.

Вместо локального Mongo (и в разработке, и на серверах) можно использовать отдельный кластер: достаточно задать в окружении полную строку подключения в `MONGO_URI`. На деплое при этом в воркфлоу укажите `mongo_enabled: false`, чтобы контейнер mongo не поднимался.

Описание параметров конфигурации сборки в GitHub Workflows

Параметры задаются в `stage-deploy.yml` и `prod-deploy.yml` и передаются в общий `reusable-deploy-config.yml`.

Inputs (with)

- **domain** — домен, на который деплоится приложение (например `app.example.com`). Используется nginx и certbot.
- **api_env** — окружение приложения: `stage`, `prod`. Подставляется в переменные и имена env-файлов.

- **env_file** — путь к env-файлу на сервере (например `.env.stage` , `.env.prod`). В него при деплое дописываются переменные из секретов.
- **nginx_mode** — режим nginx: `http` или `https` . Определяет, какой шаблон конфига и сертификаты используются.
- **certbot_test_mode** — использовать ли тестовый (staging) сервер Let's Encrypt. Полезно для проверки без лимитов.
- **migrations_run** — запускать ли миграции БД при деплое. (Этот параметр Вам не нужен, если Вы не пишете миграции для Баз данных в коде проекта)
- **blue_green_enabled** — включить ли blue/green деплой (поднятие «зелёной» копии, проверка, переключение без даунтайма (почти)).
- **deploy_mode** — способ сборки: пусто/ `default` = сборка на сервере; `registry` = сборка в CI, образ приложения пушится в GHCR и на сервере только pull.
- **node_version** — версия Node.js при сборке в Github Actions (например 24). в `.docker/Dockerfile` так же нужно прописать корректную версию nodejs, сейчас там стоит 24.13.1 версия.
- **registry_subname** — подставка в имя образа в GHCR (например `web` даёт `ghcr.io/owner/web:sha`).
- **notigy_enabled** — включить ли уведомления в Telegram о старте/успехе/ошибке деплоя. (Обязательно нужно передать токен бота, чат и топик чата по желанию). Группы и супергруппы выдают идентификатор, перед которым обязательно нужно прописать -100.
- **tag** — произвольный тег для сообщений в Telegram (например название проекта или окружения).
- **redis_enabled** — поднимать ли контейнер Redis. Если `false` , redis не стартует.

- **metrics_enabled** — поднимать ли стек метрик (Prometheus, Grafana, Loki, Telegraf, Promtail и т.д.). Если `false`, nginx не зависит от Grafana.
- **mongo_enabled** — поднимать ли контейнер MongoDB. Если `false`, ожидается внешний кластер (подключение через `MONGO_URI` в env).
- **certbot_email** — email для Let's Encrypt.
- **grafana_admin_user** и **grafana_admin_password** — логин и пароль в админку Grafana (при включённых метриках).

Secrets (секреты репозитория)

- **server_host**, **server_username**, **server_password** — хост, логин и пароль для подключения на сервер деплоя. (пока не делал ssh формат подключения)
- **env** — содержимое env-файла (переменные окружения), дописывается в `env_file` на сервере. Обычно разные секреты для stage и prod.
- **database_certificate** — опционально: сертификат/ключ для БД, если нужен отдельный файл.
- **ghcr_username**, **ghcr_token** — логин и токен для GitHub Container Registry; нужны при `deploy_mode: registry`.
- **tg_token**, **tg_chat_id**, **tg_thread_id** — токен бота, id чата и опционально id треда для Telegram-уведомлений.
- **grafana_admin_user**, **grafana_admin_password** — при желании передать из секретов вместо явных значений в `with`.

Полный список и описание параметров — в `reusable-deploy-config.yml`.

Спасибо, что дочитали до конца

Я не предлагаю универсальное решение на все случаи, но надеюсь, что этот шаблон и конфигурации помогут быстрее воплощать идеи в жизнь. Имейте в виду: сборка не рассчитана на highload, не забывайте про индексы в БД. При автодеплое иногда бывают сложности — например при пересборке уже запущенных контейнеров или при первичной выдаче сертификатов.

VPS должен быть доступен извне: проверьте, что фаервол пропускает порты 22, 80 и 443. Домен нужно купить или взять бесплатный и настроить DNS A-запись на IP сервера. В шаблоне пока нет возможности подставить уже имеющиеся у вас SSL-сертификаты — используется только выдача через Let's Encrypt.

Если шаблон пригодился — буду рад звёздочке в репозитории. Если что-то пойдёт не так — создавайте issues, постараюсь помочь. Удачи!

Теги: opensource, nextjs, boilerplate, react, fullstack

Хабы: ReactJS, Open source, GitHub, Программирование

0 ↓ 0 0 0



↑ 2 ↓

Карма

0

Общий рейтинг



Подписаться

Fedor Rychkov @StonedCatt

Frontend разработчик

Комментарии



Здесь пока нет ни одного комментария, вы можете стать первым!



Ваш комментарий



Введите текст



Отправить

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ



DazzleBizzareAdventure 7 часов назад

Похоже, Avito окончательно скатился

Простой 2 мин 8.5K

Мнение

♦ +73 10 53 +53



pilot_artem 8 часов назад

«Неплохо сделано, а?» — самолет, который обогнал звук, «Конкорд» и собственную эпоху

🗨️ Простой ⌚ 17 мин 👁 8.4K

Обзор

♦ +47 17 22 +22



sound_right 10 часов назад

Как бездумное использование AI убивает экспертизу и превращает разработку в vibe coding

🗨️ Простой ⌚ 10 мин 👁 8.6K

Мнение

♦ +34 25 41 +41



bredd_owen 5 часов назад

Как меня опрокинул автоматический скоринг Сбера

📊 Средний ⌚ 23 мин 👁 5.4K

Кейс

 +26  12  25 +25

alizar 7 часов назад

Кен Томпсон, который в одиночку написал первые версии Unix

 Простой  6 мин  6.7K

Ретроспектива

 +25  12  1 +1

TinyElectronicFriends 9 часов назад

Китайские аккумуляторы Camel и российские АКОМ. Часть 1: ёмкость 20-часового разряда

 Простой  7 мин  11K

Обзор

 +22  7  2 +2

PatientZero 4 часа назад

Мир снов, сгенерированный компьютером: виртуальная реальность для процессора 286

 12 мин  3.9K

Перевод

 +21  10  0**RationalAnswer** 11 часов назад

Военные очень хотят автономных роботов-убийц, а также паника по поводу массовых ИИ-увольнений

 9 мин  8.5K[Дайджест](#) +17  6  9 +9**BHV_publishing** 7 часов назад

От Stable Diffusion до тонкой настройки LLM: разбираем новую книгу-практикум

 Простой  7 мин  5.4K[Обзор](#) +14  17  1 +1**interpres** 3 часа назад

Запускаем Doom на старом офисном телефоне

 Средний  32 мин  3.5K[Кейс](#)[Перевод](#)

♦ +12 6 0

Сезон Heavy Digital: истории о цифровых двойниках, машинном зрении и IoT

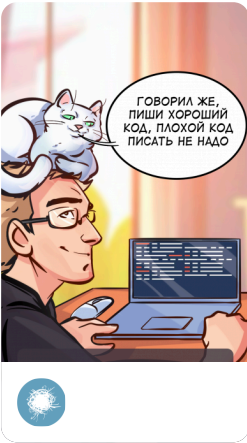
Турбо

Показать еще

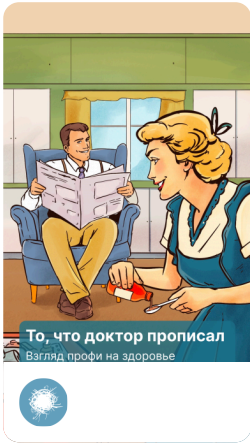
ИСТОРИИ



Лучшее из блогов компаний за неделю



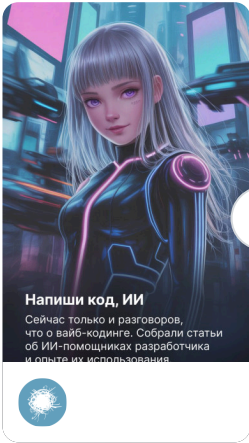
Кот в доме хозяин



Здоровье на прокачку



Схватил за мозг



Made in AI

ВОПРОСЫ И ОТВЕТЫ

React Hook Form - setValue. Какие существуют лучшие практики использования флагов shouldDirty/shouldValidate?

React · Средний · 1 ответ

Как сделать функцию "Поделиться фото в Вконтакте"?

ВКонтакте · Средний · 0 ответов

SEO оптимизация в React?

Поисковая оптимизация · Средний · 2 ответа

Долгое ожидание загрузки через браузер (Laravel + React / InertiaJS), из-за чего?

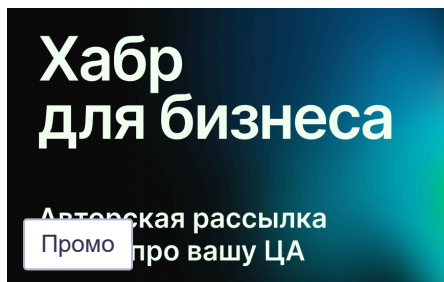
React · Простой · 0 ответов

Как сделать сложную виртуализацию?

JavaScript · Простой · 2 ответа

[Больше вопросов на Хабр Q&A](#)

МИНУТОЧКУ ВНИМАНИЯ



Подпишитесь на авторскую рассылку Хабр



Внедрил ML на заводе — опиши в новом сезоне Heavy Digital



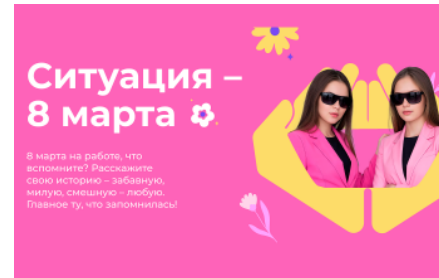
Исследуем, как айтишники относятся к ИИ в работе

БЛИЖАЙШИЕ СОБЫТИЯ

26 февраля – 6 марта

«Ситуация — 8 марта»: истории из офиса

Онлайн



Другое

Больше событий в календаре

© 2006–2026, Habr

Техническая поддержка

🌐 Настройка языка

